שלומי זוין
Boostart · CTO as a Service
shlomi@boostart.io · 054-5567213

Boostart

ספטמבר 2026 · גרסה 1.0

מתודולוגיית פיתוח מבוססת Agentic AI ו-Spec-Driven Development

פרוטוקול AI-Assisted SDLC, שערי איכות, בקרה אנושית ו-Traceability לאורך מחזור חיי הפיתוח

תוכן העניינים

1. מטרת המסמך ועקרונות יסוד
2. מחזור החיים (SDLC) מבוסס Spec
3. ארכיטקטורת הסוכנים (Multi-Agent)
4. חלוקת תפקידים בין הצוות האנושי לסוכנים
5. שערי אישור (HITL) ושער איכות
6. החוקה הפרויקטלית, Project Context ו-Skills
7. Guardrails לסוכני הפיתוח
8. ניהול זיכרון וקונטקסט
9. תיעוד, Observability ו-Traceability של עבודת הסוכנים
10. פרטיות ואבטחת מידע בעבודה עם AI
11. כלים וסביבות מאושרים ובחירת framework
12. איכות קוד שנוצר ב-AI
13. שכבת ה-AI במערכות שנמסרות
14. Best Practices לפיתוח עם AI
15. העברת ידע והטמעה במשרד

1. מטרת המסמך ועקרונות יסוד

מסמך זה מגדיר את שיטת העבודה של Boostart לפיתוח אפליקציות WEB באמצעות Agentic AI ו-Spec-Driven Development, להלן SDD. השיטה מיועדת לסביבות ממשלתיות וארגוניות שבהן נדרשים בקרה, אבטחת מידע, תיעוד, תחזוקתיות ויכולת ביקורת מלאה.

- ה-Spec הוא מקור האמת. קוד, בדיקות, תיעוד ותצורות נגזרים ממנו.
- אדם מוסמך מחזיק באחריות לכל החלטה ולכל שורת קוד שנמסרת.
- מידע אמיתי, מידע רגיש וסודות אינם מועברים למודלים מחוץ לסביבה המבודדת והמאושרת.
- השיטה Tool-agnostic. התוצרים נשמרים ב-Repository ואינם תלויים בכלי AI מסוים.
- לכל שינוי קיימת שרשרת Traceability מדרישה עסקית ועד PR, בדיקה ואישור אנושי.
- הקוד הנמסר קריא, מתועד וניתן לתחזוקה על ידי צוות המשרד גם ללא כלי AI.
- סוכן AI אינו בעל סמכות לאשר את עבודתו, לשנות סביבת ייצור או להחליף שיקול דעת מקצועי.
- תיקון קוד מתחיל בעדכון ה-Spec. אין תיקוני קוד ישירים שיוצרים פער בין הדרישה לבין המימוש.

> השימוש ב-Agentic AI נועד להאיץ עבודה מובנית ומבוקרת. הוא אינו משנה את חובת התכנון, הבדיקה, הסקירה והאישור האנושי.

2. מחזור החיים (SDLC) מבוסס Spec

שלב	מה קורה	תוצר	שער אישור
היערכות וחוקה	הגדרת חוקה פרויקטלית, כלים, הרשאות, סביבות, כללי אבטחה ו-Traceability	Constitution, Project Context, ADR ראשוני	G0
Discovery למערכת קיימת	מיפוי קוד, APIs, בסיסי נתונים, תלויות, תהליכים עסקיים ופערי תיעוד	Discovery Report, מפת רכיבים, רשימת סיכונים	אישור ארכיטקט
Spec עסקי	הגדרת מטרות, משתמשים, תרחישים, כללים, תהליכים, חריגים וקריטריוני קבלה	Requirements.md בעברית קריאה	G1
Spec עיצובי וטכנולוגי	תכנון UX, דיזיין סיסטם ממשלתי, נגישות AA, HLD/LLD, מודל נתונים, APIs ואינטגרציות	Design Spec, HLD/LLD, ADRs	G2
תכנון משימות	פירוק ה-Spec למשימות קטנות, תלויות, בדיקות ותנאי קבלה	Plan.md, Tasks.md	אישור מוביל פיתוח
מימוש	Agent ממש משימה אחת בכל Session, בענף או Worktree מבודד	קוד, בדיקות, תיעוד, PR	G3
בדיקות ו-Evals	הרצת בדיקות אוטומטיות, סריקות אבטחה, Code Review ובדיקות תרחיש	דוחות CI/CD, תוצאות Evals	שער איכות
מסירה, CI/CD ועלייה לאוויר	קידום מבוקר בין סביבות, אישור פריסה ותיעוד מסירה	Release Notes, Runbook, תיעוד תפעולי	G5
שו"ש, Spec Change	ניתוח בקשת שינוי, עדכון דרישה, תכנון השפעה ומימוש מחדש	Spec Change, עדכון Specs ומשימות	אישור לפי השפעה

בקשת שו"ש אינה נפתחת ישירות כמשימת קוד. תחילה מתועדת הסיבה לשינוי, ההשפעה על הדרישות, הנתונים, האינטגרציות, האבטחה והבדיקות.

לאחר אישור ה-Spec Change, מעודכנים Requirements.md, Plan.md ו-Tasks.md. רק לאחר מכן נפתח מימוש חדש, והבדיקות מתעדכנות בהתאם לתנאי הקבלה החדשים.

3. ארכיטקטורת הסוכנים (Multi-Agent)

סוכן	אחריות	קלט	פלט	מה אסור לו
Discovery Agent	מיפוי מערכת קיימת ותלויות	קוד, תיעוד, סכמות, לוגים מאושרים	מפת רכיבים ופערים	לשנות קוד או נתונים
Spec Agent	ניסוח והשלמת Specs	צורך עסקי, תבניות, החלטות	Requirements.md וקריטריוני קבלה	לאשר דרישות בעצמו

סוכן	אחריות	קלט	פלט	מה אסור לו
Design Agent	תכנון מסכים, זרימות ונגישות	Spec עסקי, דיזיין סיסטם	Design Spec ותסריטי UI	להחליף אישור UX אנושי
Architect Agent	HLD/LLD, APIs, נתונים ואינטגרציות	Specs, מגבלות ענן ואבטחה	תכנון טכני ו-ADRs	לשנות תשתית או IaC
Planner Agent	פירוק עבודה למשימות קטנות	Spec מאושר ותכנון טכני	Plan.md ו-Tasks.md	להחליט על עדיפויות עסקיות
Coder Agents	מימוש משימה בודדת במקביל	משימה, Context Pack, קוד רלוונטי	קוד, בדיקות וטיוטת PR	לעבוד על ענף משותף או לפרוס
Reviewer Agent	בדיקת התאמה, איכות וסיכונים	Diff, Spec, בדיקות ותיעוד	הערות Review ורשימת ממצאים	לאשר את הקוד שכתב
Test Agent	יצירת והרצת בדיקות תרחיש	תנאי קבלה, APIs, UI וקוד	Unit, API, E2E ותוצאות	לקבוע לבדו שהכיסוי מספיק
Security & License Agent	איתור חולשות, סודות ורישיונות	קוד, תלויות, SBOM	דוח אבטחה ורישוי	להחריג ממצא אבטחה
Documentation Agent	עדכון תיעוד מפתחים ותפעול	Spec, קוד מאושר, APIs	README, Runbook ותיעוד API	לתעד התנהגות שלא אושרה

כל Coder Agent עובד על משימה מוגדרת אחת בלבד, ב-branch או Worktree מבודד. עבודות מקבילות מתמזגות רק לאחר בדיקות, Code Review ואישור אנושי.

סוכן Reviewer אינו הסוכן שכתב את הקוד. סוכנים אינם מאשרים את עבודתם, אינם עוקפים שערי איכות, ואינם מבצעים פעולות קריטיות. האישור הסופי נשאר בידי גורם אנושי מוסמך.

4. חלוקת תפקידים בין הצוות האנושי לסוכנים

תפקיד אנושי	אחריות	מאשר את
מנהל מוצר מטעם המשרד	מטרות, סדרי עדיפויות, תהליכים עסקיים ותנאי קבלה	Spec עסקי, שו"ש וקבלת תוצר
ארכיטקט המשרד או ארכיטקט הענן של המערך	התאמה לחוקה ממשלתית, Nimbus, אזור נחיתה ואינטגרציות	HLD, LLD, ארכיטקטורה ושינויי ענן
מוביל Agentic SDD מטעם הספק	ניהול שיטת העבודה, Context, Agents, איכות ו-Traceability	Plan, הקצאת משימות והכנה לשערים
מפתח סוקר, Human Reviewer	סקירת קוד, הבנת לוגיקה, קריאות ותחזוקתיות	כל PR וקוד שנוצר ב-AI
DevOps	CI/CD, סביבות, הרשאות, IaC, תצפיתיות ופריסה	קידום סביבות ופעולות תשתית
נציג אבטחת מידע	אבטחת מידע, פרטיות, סודות, הרשאות ותלויות	חריגי אבטחה, מסלולי מידע ועלייה לאוויר

הסוכנים מבצעים עבודת ניתוח, ניסוח, מימוש ובדיקה. הם אינם מחליפים את בעלי התפקידים האנושיים בקבלת החלטות עסקיות, ארכיטקטוניות, אבטחתיות או תפעוליות.

5. שערי אישור (HITL) ושער איכות

שער	מתי	מי מאשר	מה נבדק
G0 חוקה ו-framework	לפני תחילת פיתוח	המשרד, ארכיטקט ענן ומוביל SDD	כלים, הרשאות, מודל עבודה, לוגים וסביבות
G1 Spec עסקי	לפני תכנון מפורט	מנהל מוצר מטעם המשרד	תרחישים, חריגים, מדדי קבלה וסדרי עדיפויות
G2 Spec עיצובי וטכנולוגי	לפני פירוק למימוש	ארכיטקט, מוצר, UX ואבטחת מידע לפי צורך	UX, AA, HLD/LLD, APIs, נתונים ואינטגרציות
G3 כל PR	לפני מיזוג לענף מאושר	Human Reviewer ומוביל פיתוח	התאמה ל-Spec, בדיקות, אבטחה, רישוי ותיעוד
G4 פעולות קריטיות	לפני ביצוע פעולה בסביבה רגישה	גורם אנושי מורשה	כתיבה ל-DB בייצור, מחיקת משאבים ושינויי IaC
G5 מסירה ועלייה לאוויר	לפני קידום לייצור	גורמי המשרד, DevOps ואבטחת מידע	תוצאות CI/CD, Runbook, ניטור, הרשאות ותוכנית חזרה לאחור

שער איכות הוא תנאי למיזוג PR ולקידום Release. תוצאות השער מתועדות במערכת ה-CI/CD, בדוח ה-PR ובשרשרת ה-Traceability.

- בדיקות אוטומטיות עוברות
- סריקת אבטחה נקייה
- התאמה מלאה ל-Spec
- ללא קוד מת
- בדיקת יעילות DB ו-API
- Traceability מלאה לדרישה
- תרחישי קבלה מכוסים
- סריקת רישיונות נקייה
- Code Review אנושי
- ללא שכפול מהותי
- תיעוד מתודה מעודכן
- אישור פריסה נדרש

כשל בשער איכות אינו נסגר באמצעות חריגה לא מתועדת. הוא מחזיר את העבודה לתיקון ב-Spec, בקוד, בבדיקות או בתצורה, לפי מקור הפער.

6. החוקה הפרויקטלית, Project Context ו-Skills

החוקה הפרויקטלית היא מסמך מחייב שנמצא ב-Repository ונקרא בתחילת כל Session של Agent. היא מגדירה את כללי העבודה שהסוכנים והמפתחים נדרשים לפעול לפיהם.

- מטרות המערכת, גבולותיה ובעלי התפקידים המאשרים.
- החוקה הממשלתית והחוקה של המשרד, לאחר קבלתן והטמעתן בפרויקט.
- כללי אבטחת מידע, פרטיות, יה"ב, הרשאות וטיפול בסודות.

- כללי פיתוח, ארכיטקטורה, naming, Code Review ותיעוד.
- מדיניות שימוש בכלי AI, מודלים, Context ונתונים.
- כללי עבודה מול Nimbus, אזור נחיתה, CI/CD ו-IaC.
- כללי נגישות AA ודיזיין סיסטם ממשלתי.
- כללי רישוי, SBOM, סריקות אבטחה וניהול תלויות.
- שערי HITL, פעולות אסורות ותנאי עצירה.
- מבנה ה-Repository, שיטת ניהול ADRs ומזהי דרישות.

Project Context הוא אוסף ממוקד של מסמכים, החלטות, קוד ודוגמאות המועבר לסוכן עבור משימה מוגדרת. הוא קטן ככל האפשר, מעודכן, ומכיל רק מידע שנדרש להשלמת המשימה באופן מדויק.

Skills הם הוראות עבודה חוזרות, גרסאות ומבוקרות, עבור משימה מקצועית מוגדרת. Skill מתאר קלט, שלבים, כללי איכות, פלט צפוי ותנאי עצירה.

Skill	שימוש
כתיבת Spec בעברית	ניסוח דרישות לפי תבנית, כולל חריגים ותנאי קבלה
דיזיין סיסטם ממשלתי	בחירת רכיבים, RTL, נגישות AA ותיעוד התנהגות
בדיקות API	יצירת בדיקות API אוטומטיות לפי חוזה ותרחישים
IaC באזור נחיתה	שינוי מבוקר של תשתית לפי כללי Nimbus והרשאות
Spec Change	ניתוח השפעה, אישור, עדכון Specs ובדיקות
Code Review	בדיקת התאמה ל-Spec, אבטחה, קריאות ותיעוד

- תבנית Requirements.md
- תבנית Design Spec
- תבנית HLD ו-LLD
- תבנית Plan.md
- תבנית Tasks.md
- תבנית ADR
- תבנית Spec Change
- תבנית PR
- תבנית Test Plan
- תבנית Runbook
- תבנית Release Notes

7. Guardrails לסוכני הפיתוח

- כל Agent פועל בהרשאת least privilege ובסביבה מבודדת.
- לסוכנים אין credentials לייצור, ואין להם גישה ישירה ל-Production DB.
- כתיבה ל-Production DB מתבצעת רק דרך Pipeline מאושר ובאישור אנושי מפורש.
- מחיקת משאבים, שינויי IaC ופעולות בלתי הפיכות מחייבים G4.
- רק כלים, פקודות, Repositories וחבילות שנכללו ב-allow-list זמינים לסוכן.

- סודות, מפתחות, tokens וקבצי credentials אינם נכנסים ל-Context של מודל.
- נתיבים רגישים, קבצי תצורה והגדרות הרשאה מוגנים מפני שינוי אוטומטי.
- תעבורת רשת יוצאת מוגבלת ליעדים ולשירותים שאושרו מראש.
- מוגדרים תקציב tokens, מגבלת זמן ומגבלת פעולות לכל משימה.
- תנאי עצירה מחייבים את הסוכן לעצור במקרה של אי-ודאות, חריגת הרשאה, כשל בדיקה או סתירה ב-Spec.
- שינוי שאינו ניתן להסבר באמצעות Spec, Diff ובדיקה אינו מתקדם למיזוג.
- אין שימוש בפקודות הרסניות ללא אישור אנושי מפורש ותיעוד הפעולה.

8. ניהול זיכרון וקונטקסט

ניהול Context הוא רכיב מרכזי בפיתוח ארוך טווח באמצעות Agents. חלון הקונטקסט של מודל אינו משמש כמקור זיכרון יחיד, ולכן הזיכרון נשמר במבנים מבוקרים ב-Repository ובמערכות מאושרות.

- ה-Repository הוא הזיכרון ארוך הטווח: החוקה, Specs, ADRs, החלטות, בדיקות, תיעוד ו-PRs.
- במערכות Legacy גדולות ניתן להקים אינדקס Vector של קוד ומסמכים לצורכי Discovery וחיפוש מבוקר.
- לכל משימה נבנה Context Pack הכולל רק את הדרישה, הקוד, החוזים וההחלטות הרלוונטיים.
- כל Session מסתיים ב-Handoff Note המתעד מה בוצע, מה נותר, אילו החלטות התקבלו ואילו סיכונים זוהו.
- החלטות ארכיטקטוניות אינן נשמרות רק ב-Chat. הן מתועדות ב-ADR בעל מזהה וסטטוס.
- מידע שהתיישן מסומן ככזה או מוסר מה-Context כדי למנוע שימוש בהנחות ישנות.

מבנה זה מונע אובדן הקשר בין Sessions, הפקת קוד על בסיס החלטות שהתבטלו, וסטייה הדרגתית בין ה-Spec לבין המימוש. הוא מאפשר לצוות אנושי להמשיך עבודה גם אם הוחלף כלי AI או מודל.

9. תיעוד, Observability ו-Traceability של עבודת הסוכנים

כל פעולת Agent מתועדת לצורך ביקורת תקציבית, אבטחתית ומקצועית. הלוגים נשמרים בסביבת הענן של המשרד, בהתאם למדיניות השמירה והגישה שתוגדר בפרויקט, לדוגמה CloudWatch ב-AWS או Cloud Logging ב-GCP.

- מזהה Session, מזהה Agent, מזהה משימה ומזהה דרישה.
- גרסת החוקה, ה-Spec, ה-Skill וה-Context Pack ששימשו לפעולה.
- Prompt שנשלח, בכפוף להסרת סודות ומידע שאסור לתעד.
- Tool calls, פקודות שבוצעו, קבצים שנקראו וקבצים ששונו.
- תשובת המודל, פלט מעובד, שגיאות ותנאי עצירה.
- ספק המודל, שם המודל, גרסה, tokens, משך פעולה ועלות.
- branch, Worktree, Diff, PR ותוצאות בדיקות קשורות.
- זהות הסוקר האנושי, החלטת האישור והסבר במקרה של דחייה.
- פעולות חריגות, ניסיונות גישה חסומים והפרות Guardrails.

מזהה דרישה → משימה → סשן סוכן → PR → סוקר אנושי → בדיקות

שרשרת זו מאפשרת לאתר במהירות מדוע נכתב רכיב מסוים, על איזה Spec הוא מבוסס, איזה Agent סייע ביצירתו, ומי קיבל את החלטת האישור. היא גם מאפשרת ניתוח עלויות AI לפי רכיב, משימה, מודל או צוות.

ניסיון בבניית מערכות multi-agent כולל תיעוד Prompts ופלטים, כללים דטרמיניסטיים לצד המודל, וזיהוי ספק המודל, ה-tokens ומשך הפעולה. עיקרון זה מיושם גם בפרוטוקול הפיתוח, בהתאמות הנדרשות למדיניות המשרד.

10. פרטיות ואבטחת מידע בעבודה עם AI

- מודלים נצרכים רק דרך חשבון הענן של המשרד, באמצעות Bedrock, Vertex AI או מסלול Nimbus מאושר אחר.
- אין הזנת מידע סודי, אישי, רגיש או מידע אמיתי לכלי SaaS חיצוני שאינו מבודד ומאושר.
- סביבות Development ו-Test אינן כוללות נתוני ייצור. הן משתמשות בנתונים סינתטיים.
- מידע אישי עובר PII redaction לפני תיעוד, אינדוקס, Eval או העברה למודל.
- קבצי secrets, מפתחות API, certificates ו-credentials אינם נכללים ב-Prompt, בלוגים או ב-Vector DB.
- כל כלי AI, Plugin, MCP Server ומודל עוברים allow-list לפני שימוש.
- נלמד ונפעל לפי מדיניות יה"ב לסוכנים, לאחר קבלתה בפרויקט, ולפי החוקה הממשלתית הרלוונטית.
- הרשאות גישה, retention של לוגים ומיקום נתונים מוגדרים יחד עם גורמי אבטחת המידע.
- פעולות Agent ניתנות לאיתור, לביקורת ולחסימה באמצעות Guardrails ותצפיתיות.
- במקרה של חשד לחשיפת מידע, ה-Agent נעצר, האירוע מתועד ומטופל לפי נוהל אירועי אבטחה של המשרד.

11. כלים וסביבות מאושרים ובחירת framework

כלי	סביבה	מתאים ל	הערות
Claude Code	סביבת פיתוח מאושרת בחשבון המשרד	עבודה Agentic, Specs, קוד, בדיקות ו-Review	הכלי שבו אנו עובדים ומציעים. המודלים נצרכים דרך Amazon Bedrock או Vertex AI, בכפוף לאישור Nimbus.
Kiro, Antigravity, Spec-kit, BMAD	לפי אישור המשרד ו-Nimbus	תאימות ארטיפקטים	המכרז מפרט כלים אלה. הארטיפקטים נשמרים ב-Markdown במבנה Requirements, Plan ו-Tasks.

ההחלטה על framework מתקבלת ומתועדת בשער G0 יחד עם המשרד, ארכיטקט הענן וגורמי אבטחת המידע. הבחירה תהיה רק מתוך כלים המאושרים לעבודה ב-Nimbus ובסביבה הספציפית של הפרויקט.

הארטיפקטים נשמרים כקבצים קריאים ב-Repository: Constitution, Requirements.md, Plan.md, Tasks.md, ADRs, בדיקות ותיעוד. הם תואמים למבנה Requirements, Plan ו-Tasks שבו משתמשות המסגרות המפורטות במכרז, ולכן המשרד אינו נעול ל-Claude Code ויכול להמשיך בכל כלי מאושר בהמשך.

12. איכות קוד שנוצר ב-AI

בדיקות

כל מימוש כולל בדיקות הנגזרות מקריטריוני הקבלה ב-Spec. הבדיקות נכתבות ומורצות ב-CI/CD, אך התרחישים, מקרי הקצה והמשמעות העסקית שלהם מאושרים גם על ידי גורם אנושי.

- Unit tests ללוגיקה, כללים, המרות וחריגים.
- API tests אוטומטיים בסגנון Postman או כלי מקביל, לפי חוזי API.
- UI ו-E2E tests לתרחישים עסקיים מרכזיים.

- Visual tests לרכיבי UI, RTL, דיזיין סיסטם ונגישות.
- Integration tests לאינטגרציות, תורים, תהליכים אסינכרוניים והרשאות.
- Regression tests לכל תקלה שתוקנה ולכל שו"ש משמעותי.

כיסוי אינו נמדד רק כאחוז שורות קוד. הוא נמדד גם לפי תרחישים עסקיים, חריגים, הרשאות, מעברי מצב, כשלי אינטגרציה ומקרי קצה. Human Reviewer בודק במיוחד שבדיקות שנוצרו ב-AI אינן בודקות רק את ההתנהגות שהקוד עצמו מימש.

חולשות אופייניות לקוד AI

קוד שנוצר על ידי מודל עלול לכלול API מיושן, תלות לא מתועדת, טיפול חסר בשגיאות, לוגיקה שנראית תקינה אך נכשלת במקרי קצה, או שימוש בלתי בטוח בקלט משתמש. לכן נדרשת בדיקה ייעודית מעבר להרצת tests.

- שימוש ב-API מיושן
- SQL או command injection
- הרשאה חסרה או רחבה
- טיפול שגיאות חלקי
- לוגיקה סמויה לא מתועדת
- תלות לא מאושרת
- קוד שלא נקרא
- שכפול קוד גנרטיבי
- race conditions
- ולידציה חסרה לקלט

Reviewer Agent מסמן ממצאים, אך Human Reviewer מחויב להבין את הלוגיקה, את השפעתה ואת התיקון. קוד שלא ניתן להסבר על ידי הסוקר אינו מאושר.

יעילות ועלות ענן

בכל PR נבדקת יעילות בסיסית של גישות למסד הנתונים, APIs, cache, streaming, פעולות רקע ושימוש במשאבי ענן. Agentic AI עלול לייצר קריאות חוזרות או שכבות מיותרות שנראות תקינות פונקציונלית אך מגדילות עלויות ותלות.

נבדקים בין היתר שאילתות ללא אינדקס, N+1 queries, polling מיותר, קריאות API כפולות, טעינת מידע עודף, retries לא מבוקרים ולולאות ללא גבול. ממצאים מהותיים נרשמים ב-PR או ב-ADR.

תחזוקתיות ותיעוד ברמת המתודה

נבדקים קריאות קוד, הפרדת אחריות, naming, גבולות בין שכבות, טיפול בשגיאות, testability, ומדדי תחזוקתיות שייקבעו בפרויקט. אין להשאיר קוד מת, פיצול לא מוסבר של לוגיקה, או שכפול שנוצר כתוצאה מעבודה מקבילית של Agents.

מתודות ציבוריות, חוזי API, כללים עסקיים ותהליכים מורכבים מתועדים ברמת המתודה או הרכיב, כאשר התיעוד מוסיף הסבר שאינו מובן מאליו מהקוד. התיעוד מתעד את ה"למה", ולא רק את ה"מה".

זכויות יוצרים ורישוי

כל תלות חדשה עוברת בדיקת רישוי. קוד ותלויות בעלי רישיון מגביל כגון GPL או AGPL אינם נכנסים לפרויקט ללא החלטה מפורשת של המשרד והגורמים המוסמכים.

מופק SBOM כחלק מתהליך המסירה וה-Release. סריקת רישוי ב-CI/CD בודקת גם תלות טרנזיטיבית, גרסאות פגיעות והכנסת חבילות לא מאושרות.

13. שכבת ה-AI במערכות שנמסרות

כאשר המערכת הנמסרת כוללת Agents או יכולות GenAI, שכבת ה-AI מתוכננת כרכיב מערכת מבוקר ולא כאוסף קריאות ישירות למודל. היא כוללת הפרדה בין לוגיקה עסקית, ספקי מודלים, מידע, הרשאות, מדידה ובקרה.

AI Gateway

- ניתוב מודלים לפי משימה, מדיניות, עלות, זמינות ורגישות מידע.
- fallback ו-retry מבוקרים, עם מגבלות למניעת לולאות ועלויות בלתי צפויות.
- ניהול מפתחות והרשאות ללא חשיפתם ללקוח או ל-Agent.
- rate limits לפי משתמש, שירות, Agent או תהליך.
- caching כאשר התשובה מותרת לשימוש חוזר ואינה כוללת מידע אישי.
- token tracking, עלות לפי בקשה, תקציב והתראות.
- PII redaction לפני שליחה למודל ולפני שמירת לוגים.
- Tracing של בקשות, מודלים, כלים, Latency, עלות ותוצאות.

MCP Gateway/Registry ו-Guardrails

MCP Gateway או Registry מרכזים את הגישה לכלים חיצוניים. כל MCP Server נרשם, עובר בדיקת אבטחה, מוגדר עם הרשאות מצומצמות, ומנוהל לפי allow-list.

Guardrails נאכפים לפני ואחרי קריאת המודל. הם כוללים בדיקת הרשאות, סינון מידע רגיש, הגבלת כלי פעולה, ולידציה של פלט מובנה, כללים עסקיים, חסימת הוראות אסורות וניתוב לאישור אנושי במקרים נדרשים.

Eval Suite-ו Evals

Eval Suite כולל Test Dataset גרסאי, הכולל תרחישים רגילים, מקרי קצה, קלטים בעייתיים, שיחות מדומות ודוגמאות מאושרות. כל שינוי ב-Spec, Prompt, Agent, מודל או כלי מחייב הרצת Evals רלוונטיים.

מדד	הגדרה	שימוש
דיוק	שיעור תשובות תואמות אמת או מקור מאושר	בדיקת איכות תשובה
עמידה בכללים	התאמה לכללים עסקיים ולמדיניות	חסימת התנהגות אסורה
שיעור הזיה	מידע שלא נתמך במקור או בתהליך	איתור סטייה עובדתית
שיעור סטייה	חריגה מה-Spec או מהפורמט הנדרש	בדיקת נאמנות לדרישה
הצלחת כלי	הצלחת קריאות Tool או MCP	בדיקת תהליך Agent
זמן תגובה	Latency לפי תרחיש ומודל	תכנון חוויית משתמש
עלות לתרחיש	tokens ועלות קריאה מלאה	FinOps ובחירת מודל
שיעור escalation	העברה לאדם או לתהליך חלופי	כיול Guardrails

Evals משמשים Quality Gate ב-CI/CD עבור רכיבי AI. שינוי שאינו עומד בספי האיכות שנקבעו לפרויקט אינו מתקדם לפרסום או לייצור ללא החלטה אנושית מתועדת.

Specs, Prompts, Agents וכללי Guardrails הם גרסאות מנוהלות. פרסום שינוי הוא פעולה נפרדת משמירה. ניתן לשמור Draft, לבצע Review, להריץ Evals, ורק לאחר אישור לבצע publish לגרסה פעילה.

FinOps כולל תקציבי tokens, מגבלות עלות, התראות על חריגה, דוחות שימוש ו-kill-switch להשבתת Agent, כלי, מודל או מסלול פעולה. פעולות בעלות השפעה גבוהה מוגדרות מראש עם fallback אנושי.

ניסיון בבניית מערכות multi-agent כולל תפקידים מופרדים, Agents משורשרים, Context מופעל, מערכות זיכרון, RAG, כללים דטרמיניסטיים לצד מודל, תיעוד Prompts ופלטים, וניתוב בין מספר ספקי מודלים.

14. Best Practices לפיתוח עם AI

- מפרקים עבודה למשימות קטנות וברורות.
- מקצים משימה אחת לכל Session של Agent.
- כותבים קריטריוני קבלה לפני כתיבת קוד.
- מצרפים לכל משימה Context Pack ממוקד.
- לא מקבלים קוד שלא ניתן להסביר ולתחזק.
- לא נותנים ל-Agent לאשר את עבודתו.
- מעדכנים Spec לפני כל תיקון מהותי.
- מריצים בדיקות לפני Code Review.
- בודקים מקרי קצה אנושיים לבדיקות שנוצרו ב-AI.
- שומרים החלטות ב-ADR ולא רק בשיחה.
- משתמשים ב-branch או Worktree מבודד לכל עבודה מקבילה.
- חוסמים סודות, נתונים אמיתיים ופעולות ייצור מה-Context.
- מודדים עלות, tokens ומשך פעולה של Agents.
- מעדכנים תיעוד יחד עם הקוד ולא לאחר המסירה.
- מבצעים publish נפרד משמירת Draft ברכיבי AI.
- מעדיפים כללים דטרמיניסטיים כאשר ניתן להגדיר כלל עסקי מפורש.

15. העברת ידע והטמעה במשרד

העברת הידע מתבצעת תחילה בהדרכה פרונטלית לצוותי המשרד והספקים המעורבים, בהתאם להעדפת המכרז. לאחר מכן היא נמשכת בעבודה משותפת על Specs, PRs, שערי איכות ותרחישי מסירה.

נושא הדרכה	אופן העברה	תוצר
מבנה ה-Spec	הדרכה ועבודה על דוגמה מהפרויקט	תבניות ו-Spec מאושר
אישור Specs	סימולציית G1 ו-G2	רשימת בדיקות מאשרים
סקירת PR שנוצר ב-AI	Code Review מודרך	Checklist לסוקר אנושי
ניהול Project Context	עבודה על Context Pack ו-Handoff	נוהל Context ו-ADRs
הפעלת CI/CD ושערים	מעבר על Pipeline אמיתי	Runbook ושגרת Release
ניהול Agents במערכת	הדרכה על גרסאות, Evals ו-publish	נוהל תפעול Agent

נושא הדרכה	אופן העברה	תוצר
אבטחת מידע ופרטיות	עבודה מול גורמי המשרד	כללי שימוש והסלמה
שו"ש מבוסס Spec	תרגול בקשת שינוי מלאה	תבנית Spec Change

התוצרים כוללים חוקה פרויקטלית, תבניות, Skills, מדריכי עבודה, Runbooks, דוגמאות PR, רשימות בדיקה, ADRs ותיעוד תפעולי. כל תוצר נשמר ב-Repository של הפרויקט ומנוהל בגרסאות.

במשרד שבו טרם קיימים תהליכי AI מוסדרים, נתחיל בהכנת תשתית העבודה: מיפוי בעלי תפקידים, בחירת סביבה מאושרת, הגדרת הרשאות, לוגים, מדיניות נתונים, כללי HITL, תהליך אישור כלים והקמת תבניות עבודה. המטרה היא לאפשר לצוות המשרד להמשיך לתחזק, לפתח ולבקר את המוצר באופן עצמאי.

שלומי זוין

Boostart · CTO as a Service

shlomi@boostart.io · 054-5567213